MANAKULA VINAYAGAR
INSTITUTE OF TECHNOLOGY
An Autonomous Institution
Affiliated to Pondicherry University, Approved by AICTE, New Delhi,
Accredited by NBA, New Delhi and NAAC with 'A' Grade
Kalitheerthalkuppam, Puducherry- 605 107.

A Consumer-Side Carbon-Aware Routing and Transparency Auditing Framework for LLM API Requests

Team Members:

1.KATHIRAVAN A

2.KISHOARE V

3. KRISHNAKUMAR G

4. GOKULAKRISHNAN D

Project Guide:

Mrs. VIMALA DHEEKSHANYA

ASSISTANT PROFESSOR/ IT

INDEX

ABSTRACT

Large Language Model (LLM) inference is increasingly embedded in consumer and developer-facing applications, yet the environmental cost of each individual API call remains invisible to the developers who trigger it. Existing carbon-aware routing systems (e.g., GAR, Green SERV) operate at the infrastructure/provider level and are not usable by independent developers calling public commercial APIs such as OpenAI, Anthropic, or Grok. Existing carbon-tracking products (e.g., carbon-llm) report emissions but trust the provider's self-reported model and token usage without verification. This project proposes Eco-Query, a lightweight, developer-facing proxy and SDK that (i) classifies each incoming query by required capability tier, (ii) estimates the carbon footprint of serving that query on each candidate model using published energy-per-token figures and live grid carbon-intensity data, (iii) routes the request to the lowest-carbon model that still meets a minimum quality and latency requirement, and (iv) verifies, via response latency and token-throughput fingerprinting, whether the provider actually served the model it claims to have used, flagging inconsistencies in the audit trail. A web dashboard visualises cumulative carbon savings, per-model usage, and verification flags.

The system is evaluated on a small multi-provider model pool across standard NLP benchmark subsets, comparing macro accuracy and estimated CO2 per request against always-largest-model and always-cheapest-model baselines, following the evaluation methodology established in prior carbon-aware routing literature. The contribution of this work is not the routing algorithm itself, which builds on established techniques, but the combination of consumer-side deploy ability with an independent, carbon-linked provider-integrity verification layer, which is absent from existing academic and commercial systems.

INTRODUCTION

Large Language Models (LLMs) are now called through commercial APIs (OpenAI, Anthropic, Grok, and others) by millions of developers building consumer applications, internal tools, and automation pipelines. Each API call consumes electricity at the provider's data centre, and that electricity carries an associated carbon footprint that varies significantly depending on the model size invoked, the data centre's regional grid mix, and the time of day the request is served. Prior work has shown that inference now accounts for more than half of the total lifecycle carbon emissions of large language models, and that grid carbon intensity can vary by an order of magnitude across regions and time windows.

Despite this, the developer calling these APIs today has no mechanism to (a) choose a lower-carbon model automatically when a smaller model would suffice, (b) see an estimate of the carbon cost of their own usage, or (c) verify that the provider actually served the model it billed for. Existing carbon-aware routing research operates inside the infrastructure of the LLM provider itself and is not something an external developer can deploy. Existing carbon tracking tools accept the provider's self-reported usage at face value. This project addresses this specific, currently unfilled gap.

PROBLEM STATEMENT

Independent developers integrating LLM APIs into their applications have no accessible mechanism to (1) automatically route each request to the lowest-carbon model that still meets their quality requirements, (2) obtain a trustworthy, per-request estimation of the carbon footprint of their AI usage, or (3) verify that the provider served the model it claims to have billed for. This project designs and implements a system that addresses all three limitations as a self-contained, deployable software layer.

OBJECTIVES

- To design a lightweight query-classification mechanism that determines the minimum model capability tier required for a given prompt.
- To design a carbon estimation model that combines published per-model energy-per-token figures with live regional grid carbon-intensity data.
- To implement a router that selects the lowest-estimated-carbon model from a multi-provider pool subject to a quality/latency floor.
- To implement a provider-integrity verification mechanism using response latency and token-throughput fingerprinting.
- To build an audit logging system and a web dashboard visualising cumulative carbon usage, per-model breakdown, and verification flags.
- To evaluate the system on standard NLP benchmark subsets, comparing accuracy and estimated carbon against baseline routing strategies.

PROPOSED SYSTEM ARCHITECTURE

The system consists of five logical modules: the Query Classifier, the Carbon Estimator, the Router, the Verification Engine, and the Audit/Dashboard layer. A client application (or the project's own demo chat interface) sends a request to the Eco-Query backend, which performs classification, estimation, and routing before forwarding the request to the selected provider's real API. The response, together with its actual token usage and latency, is logged and checked against the expected behavioural profile of the claimed model.

WORK FLOW

- Client sends prompt to Eco-Query backend.
- Query Classifier tags the prompt with a required capability tier (simple / medium / complex).
- Carbon Estimator computes an estimated CO2 cost for every model in the pool that meets the required tier, using energy-per-token figures and live grid intensity for that provider's known region.
- Router selects the feasible model with the lowest estimated carbon; ties are broken by lower latency.
- Backend calls the selected provider's real API and receives the response.
- Verification Engine compares observed latency and token throughput against the expected profile for the claimed model and flags anomalies.
- Audit Logger writes the full record (model, tokens, estimated carbon, latency, verification flag) to the database.
- Dashboard queries the database and renders cumulative and per-model carbon statistics.

METHODOLOGY AND IMPLEMENTATION

Technological Stack:

Layer	Technology
Backend / Proxy	FastAPI (Python) / Node.js + Express
Frontend	React.js + Recharts or Streamlit
Database	MongoDB
LLM Providers	OpenAI API, Anthropic API, Grok API, Ollama
Grid Carbon Data	Electricity Maps API / WattTime API

MODULE DESCRIPTION

Module	Responsibility	Key Output
Query Classifier	Determine minimum required model capability tier for a prompt	Tier label (simple/medium/complex)
Carbon Estimator	Compute estimated CO2 for each feasible model	Ranked list of (model, estimated carbon)
Router	Select lowest-carbon feasible model; call provider API	Chosen model + API response
Verification Engine	Detect provider behaviour inconsistent with claimed model	Verification flag + confidence rating
Audit Logger	Persist every request's full record	MongoDB document per request
Dashboard	Visualise cumulative and per-model carbon data	Charts: total CO2, per-model split, flags over time

LITERATURE SURVEY

The following works were reviewed to establish the state of the art in carbon-aware LLM inference and routing, and to identify the specific gap this project addresses.

#	Paper / System	Year	Contribution	Limitation w.r.t. this project
1	Ozcan et al., "Quantifying the Energy Consumption and Carbon Emissions of LLM Inference via Simulations" (Vidur + Vessim)	2025	GPU power model + grid co-simulation for LLM inference energy estimation	Simulation-only; not a deployable routing system
2	Fu, Fan, Jiang, "CO2-Meter"	2025	Operational + embodied carbon estimator for LLMs on edge SoCs	Targets edge hardware, not cloud API consumers

3	Ong et al., "RouteLLM"	2024	Preference-based LLM routing	No carbon awareness
4	Sheshanarayana et al., "GAR"	2026	Carbon-aware constrained routing	Provider-side only
5	Patterson et al.	2021	Carbon emissions of large models	Training-focused, not inference routing

RESEARCH GAP

No existing academic system or commercial product combines the following three properties:

(i) Deployability by an individual developer against unmodified public commercial LLM APIs, without requiring control of the underlying serving infrastructure;

(ii) Carbon-aware routing across multiple independent providers rather than within a single provider's internal model pool.

(iii) Verification that the provider's claimed model matches its observed behaviour, with that verification feeding back into the carbon estimate. Eco-Query is positioned to fill this specific combination, while explicitly building on the routing formulation established by GAR and the auditing philosophy established by Gate Scope.

While GAR and RouteLLM focus on performance-cost trade-offs, and simulation works like Özcan et al. focus on energy modeling, our work uniquely combines consumer-side deployability, multi-provider routing, and independent provider verification in a single lightweight proxy

CONCLUSION

This project proposes Eco-Query, a consumer-side carbon-aware routing and transparency auditing framework for LLM API requests. While carbon-aware routing and carbon tracking have both been addressed independently in recent literature and commercial products, no existing system combines multi-provider, developer-deployable routing with an independent verification layer that ties provider integrity checks to carbon accounting. Eco-Query is

positioned as a modest but well-defined contribution within an active and fast-moving research area, building explicitly on the routing formulation of GAR and the auditing philosophy of Gate Scope, while targeting a deployment context — independent developers using unmodified public APIs — that existing work does not address.

REFERENCES

[1] Isaac Ong et al. "RouteLLM: Learning to Route LLMs with Preference Data." arXiv:2406.18665, 2024.

[2] Miray Özcan et al. "Quantifying the Energy Consumption and Carbon Emissions of LLM Inference via Simulations." arXiv:2507.11417, 2025.

[3] Zhenxiao Fu, Chen Fan, Lei Jiang. "CO2-Meter: A Comprehensive Carbon Footprint Estimator for LLMs on Edge Devices." arXiv:2511.08575, 2025.

[4] Disha Sheshanarayana et al. "GAR: Carbon-Aware Routing for LLM Inference via Constrained Optimization." arXiv:2605.11603, 2026.

[5] Johnny R. Zhang et al. "Carbon-Aware Compute–Power Scheduling for AI Data Centers with Microgrid Prosumer Operations." arXiv:2605.03751, 2026.

[6] David Patterson et al. "Carbon Emissions and Large Neural Network Training." arXiv:2104.10350, 2021.

[7] Alexandre Lacoste et al. "Quantifying the Carbon Emissions of Machine Learning." arXiv:1910.09700, 2019.

[8] Emma Strubell et al. "Energy and Policy Considerations for Deep Learning in NLP." ACL 2019.

[9] Sasha Luccioni et al. "Power Hungry Processing: Watts Driving the Cost of AI Deployment?" arXiv:2311.16863, 2023.